

게임 수학

강의 9: 복소수와 쿼터니언

복소수와 2D 회전, 쿼터니언과 3D 회전

동명대학교 게임공학과
강영민

2026년 1학기

Contents

1	복소수 복습	2
1.1	복소수의 기하적 표현 — 복소 평면	2
2	복소수 연산과 2D 회전	2
2.1	덧셈과 뺄셈	2
2.2	복소수 곱셈	3
2.3	복소수 곱셈의 행렬 표현	3
3	쿼터니언 (Quaternion)	3
3.1	쿼터니언의 탄생	3
3.2	쿼터니언의 기본 연산	4
3.3	쿼터니언 곱셈	4
4	쿼터니언으로 3D 회전	5
4.1	쿼터니언 연속 회전	5
4.2	단위 쿼터니언의 성질	5
5	쿼터니언과 회전 행렬의 변환	6
5.1	쿼터니언 $\rightarrow$ 회전 행렬	6
5.2	회전 행렬 $\rightarrow$ 쿼터니언	6
6	SLERP — 구면 선형 보간	6
7	오일러 각도 vs 쿼터니언	7
8	Python 구현	7
9	연습 문제	10
10	요약	11
A	오일러 공식 유도	11
B	쿼터니언 곱셈 전개	11

1 복소수 복습

정의 — 복소수 (Complex Number)

복소수는 실수부(real part)와 허수부(imaginary part)로 구성된 수:

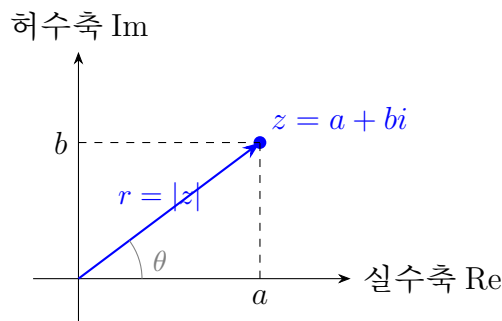
$$z = a + bi, \quad a, b \in \mathbb{R}$$

여기서 i 는 허수 단위(imaginary unit): $i^2 = -1$.

- $\operatorname{Re}(z) = a$ (실수부)
- $\operatorname{Im}(z) = b$ (허수부)
- $|z| = \sqrt{a^2 + b^2}$ (절댓값/크기)
- $\bar{z} = a - bi$ (켈레 복소수/복소 켈레)

1.1 복소수의 기하적 표현 — 복소 평면

복소수 $z = a + bi$ 는 2D 평면의 점 (a, b) 로 표현할 수 있다. 이를 복소 평면(complex plane) 또는 아르간 평면(Argand plane)이라 한다.



극형식 (polar form):

$$z = r(\cos \theta + i \sin \theta) = re^{i\theta}$$

여기서 $r = |z|$ 는 크기, $\theta = \arg(z)$ 는 편각(argument).

오일러 공식.

$$e^{i\theta} = \cos \theta + i \sin \theta$$

특히 $e^{i\pi} + 1 = 0$ (오일러 등식).

2 복소수 연산과 2D 회전

2.1 덧셈과 뺄셈

$$(a + bi) + (c + di) = (a + c) + (b + d)i$$

기하적으로: 벡터 덧셈과 동일.

2.2 복소수 곱셈

복소수 곱셈

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$$

(분배법칙 적용 후 $i^2 = -1$ 대입)

기하적 의미: 극형식으로 보면 복소수 곱셈은 크기는 곱하고, 각도는 더한다:

$$r_1 e^{i\theta_1} \cdot r_2 e^{i\theta_2} = r_1 r_2 e^{i(\theta_1 + \theta_2)}$$

정리 — 복소수 곱셈은 회전+스케일

단위 복소수 $q = \cos \theta + i \sin \theta$ ($|q| = 1$)를 복소수 z 에 곱하면, z 를 원점 중심으로 θ 만큼 회전시킨다:

$$z' = q \cdot z$$

예제 1 — 복소수로 2D 회전

점 $(1, 0)$ 을 90° 회전하라.

$$z = 1 + 0i = 1$$

$$\text{회전 복소수: } q = \cos 90^\circ + i \sin 90^\circ = 0 + 1 \cdot i = i$$

$$z' = q \cdot z = i \cdot 1 = i = 0 + 1i$$

결과: $(0, 1)$ ✓ ((1, 0)을 90° 회전하면 $(0, 1)$)

예제 2 — 두 회전의 합성

30° 회전 후 45° 회전 = 총 75° 회전임을 복소수로 확인하라.

$$q_1 = e^{i \cdot 30^\circ} = \cos 30^\circ + i \sin 30^\circ$$

$$q_2 = e^{i \cdot 45^\circ} = \cos 45^\circ + i \sin 45^\circ$$

$$q_1 q_2 = e^{i(30^\circ + 45^\circ)} = e^{i \cdot 75^\circ} = \cos 75^\circ + i \sin 75^\circ$$

합성 회전 = 75° ✓

2.3 복소수 곱셈의 행렬 표현

복소수 곱셈 $z' = (a + bi)(x + yi)$ 를 행렬로 쓰면:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & -b \\ b & a \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

$|a + bi| = 1$ 이면 이 행렬이 2D 회전 행렬이 된다.

3 쿼터니언 (Quaternion)

3.1 쿼터니언의 탄생

복소수가 2D 회전을 우아하게 표현하듯이, 3D 회전을 위해 확장된 수 체계가 필요하다. 수학자 해밀턴(Hamilton)은 1843년에 쿼터니언(quaternion)을 발명했다.

정의 — 쿼터니언

쿼터니언은 하나의 실수부와 세 개의 허수 단위 i, j, k 로 구성된 수:

$$q = w + xi + yj + zk, \quad w, x, y, z \in \mathbb{R}$$

또는 스칼라 + 벡터 형태:

$$q = (w, \mathbf{v}), \quad \mathbf{v} = (x, y, z)^T$$

세 허수 단위의 규칙:

$$i^2 = j^2 = k^2 = ijk = -1$$

이로부터:

$$\begin{aligned} ij &= k, & jk &= i, & ki &= j \\ ji &= -k, & kj &= -i, & ik &= -j \end{aligned}$$

쿼터니언 곱셈은 가위곱과 연결

i, j, k 순환 규칙은 벡터 가위곱의 표준 기저 규칙과 동일하다: $\mathbf{i} \times \mathbf{j} = \mathbf{k}, \mathbf{j} \times \mathbf{k} = \mathbf{i}, \mathbf{k} \times \mathbf{i} = \mathbf{j}$. 이것은 우연이 아니다.

3.2 쿼터니언의 기본 연산

쿼터니언 연산

$p = (p_w, \mathbf{p}), q = (q_w, \mathbf{q})$ 에 대해:

덧셈: $p + q = (p_w + q_w, \mathbf{p} + \mathbf{q})$

스칼라 배: $kq = (kq_w, k\mathbf{q})$

켈레: $q^* = (q_w, -\mathbf{q})$

크기: $|q| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2}$

역: $q^{-1} = q^*/|q|^2$ (단위 쿼터니언이면 $q^{-1} = q^*$)

3.3 쿼터니언 곱셈

쿼터니언 곱셈

$p = (p_w, \mathbf{p}), q = (q_w, \mathbf{q})$ 의 쿼터니언 곱:

$$pq = (p_w q_w - \mathbf{p} \cdot \mathbf{q}, \quad p_w \mathbf{q} + q_w \mathbf{p} + \mathbf{p} \times \mathbf{q})$$

실수부: 스칼라 곱을 빼 것, 허수부: 선형 결합 + 가위곱.

쿼터니언 곱셈의 특성.

- 결합법칙: $(pq)r = p(qr)$ ✓
- 교환법칙 불성립: $pq \neq qp$ (가위곱 항 때문)
- 단위원소: $q \cdot 1 = 1 \cdot q = q$, 여기서 $1 = (1, \mathbf{0})$

4 쿼터니언으로 3D 회전

정리 — 쿼터니언 회전

단위 벡터 $\hat{\mathbf{n}}$ 을 회전축으로 각도 θ 만큼 회전시키는 단위 쿼터니언:

$$q = \cos \frac{\theta}{2} + \hat{\mathbf{n}} \sin \frac{\theta}{2} = \left(\cos \frac{\theta}{2}, \hat{\mathbf{n}} \sin \frac{\theta}{2} \right)$$

3D 점(또는 벡터) $\mathbf{v}$ 를 회전시키려면, 순수 쿼터니언 $p = (0, \mathbf{v})$ 에 대해:

$$p' = qpq^{-1} = qpq^*$$

(단위 쿼터니언이므로 $q^{-1} = q^*$)

p' 의 벡터 부분이 회전된 벡터 $\mathbf{v}'$ 이다.

왜 $\theta/2$ 인가? 쿼터니언 곱 qpq^{-1} 을 전개하면 실제로 θ 만큼 회전됨을 보일 수 있다. 여기서 각도가 두 번(q 와 q^{-1} 양쪽에서) 작용하므로 $\theta/2$ 씩 나눈다.

예제 3 — 쿼터니언으로 3D 회전

벡터 $(1, 0, 0)$ 을 z 축 기준으로 90° 회전하라.

회전축: $\hat{\mathbf{n}} = (0, 0, 1)$, 각도 $\theta = 90^\circ$

$$q = \cos 45^\circ + (0, 0, 1) \sin 45^\circ = \left(\frac{\sqrt{2}}{2}, 0, 0, \frac{\sqrt{2}}{2} \right)$$

$p = (0, 1, 0, 0)$ (순수 쿼터니언)

$$q^* = \left(\frac{\sqrt{2}}{2}, 0, 0, -\frac{\sqrt{2}}{2} \right)$$

$p' = qpq^*$ 를 전개하면 $p' = (0, 0, 1, 0)$

결과 벡터: $(0, 1, 0) \checkmark ((1, 0, 0)$ 을 z 축 90° 회전 = $(0, 1, 0)$)

4.1 쿼터니언 연속 회전

두 회전 q_1 과 q_2 를 순서대로 적용:

$$\mathbf{v}'' = q_2(q_1\mathbf{v}q_1^*)q_2^* = (q_2q_1)\mathbf{v}(q_2q_1)^*$$

즉, 합성 회전 쿼터니언 = q_2q_1 (행렬 곱처럼 역순).

4.2 단위 쿼터니언의 성질

단위 쿼터니언 성질

$|q| = 1$ 인 단위 쿼터니언에 대해:

1. $q^{-1} = q^*$ (역 = 켤레)
2. $-q$ 는 q 와 동일한 회전을 나타낸다 ($-q$ 는 $\theta + 360^\circ$ 회전과 같음).
3. 단위 쿼터니언의 집합은 4차원 단위 구면 S^3 을 이룬다.

4. q 와 $-q$ 는 같은 3D 회전 $\Rightarrow$ $SO(3)$ 에서 S^3 으로의 2:1 커버링 사상.

5 쿼터니언과 회전 행렬의 변환

5.1 쿼터니언 $\rightarrow$ 회전 행렬

쿼터니언에서 회전 행렬

단위 쿼터니언 $q = (w, x, y, z)$ 에 해당하는 3×3 회전 행렬:

$$R = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{pmatrix}$$

5.2 회전 행렬 $\rightarrow$ 쿼터니언

역변환(Shepperd's method):

$$w = \frac{\sqrt{1 + R_{11} + R_{22} + R_{33}}}{2}$$

나머지 성분:

$$x = \frac{R_{32} - R_{23}}{4w}, \quad y = \frac{R_{13} - R_{31}}{4w}, \quad z = \frac{R_{21} - R_{12}}{4w}$$

(단, w 가 작을 때는 다른 방법 사용)

6 SLERP — 구면 선형 보간

정의 — SLERP (Spherical Linear Interpolation)

두 단위 쿼터니언 q_1, q_2 사이를 일정한 각속도로 보간:

$$\text{slerp}(q_1, q_2, t) = q_1 (q_1^{-1} q_2)^t, \quad t \in [0, 1]$$

또는 동치 공식:

$$\text{slerp}(q_1, q_2, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_1 + \frac{\sin(t\Omega)}{\sin \Omega} q_2$$

여기서 $\cos \Omega = q_1 \cdot q_2$ (4D 내적).

왜 SLERP인가? 일반 선형 보간(LERP)을 쿼터니언에 적용하면 크기가 1이 아니게 되어 왜곡이 생긴다. SLERP는 구면 위에서 보간하여 항상 단위 쿼터니언을 반환하고, 각속도가 일정하다.

게임에서 사용 예:

- 캐릭터 회전 애니메이션 블렌딩

- 카메라 부드러운 방향 전환
- IK(역운동학) 솔버

7 오일러 각도 vs 쿼터니언

특성	오일러 각도	쿼터니언
표현	3개의 수 (α, β, γ)	4개의 수 (w, x, y, z)
짐벌 락	있음	없음
보간	부자연스러움	SLERP으로 자연스러움
합성	행렬 필요	단순 곱셈
역변환	복잡	켈레 q^*
정규화	불필요	필요 ($ q = 1$ 유지)
직관성	높음 (편집 UI용)	낮음 (내부 계산용)

실무에서의 사용

- 내부 표현: 쿼터니언 (계산 효율, 짐벌 락 없음)
- 에디터/인터페이스: 오일러 각도 (직관적으로 편집 가능)
- 애니메이션: SLERP 쿼터니언 보간
- Unity, Unreal, Three.js 모두 내부적으로 쿼터니언 사용

8 Python 구현

실습

```

1 import numpy as np
2 import cmath # 복소수
3
4 # =====
5 # 1. 복소수와 2D 회전
6 # =====
7 # 복소수  $z = 1 + 0i$  (축  $x$  단위벡터)
8 z = complex(1, 0)
9
10 # 도 90 회전 쿼터니언 복소수 ( )
11 theta = np.radians(90)
12 q = complex(np.cos(theta), np.sin(theta))

```

```

13
14 z_rot = q * z
15 print(f"도90 회전: ({z.real},{z.imag}) -> ({z_rot.real:.4f},{
16     z_rot.imag:.4f})")
17 # (1,0) -> (0.0, 1.0)
18
19 # 두회전합성
20 q1 = complex(np.cos(np.radians(30)), np.sin(np.radians(30)))
21 q2 = complex(np.cos(np.radians(45)), np.sin(np.radians(45)))
22 q_combined = q2 * q1
23 angle_combined = np.degrees(cmath.phase(q_combined))
24 print(f"도30+45 합성: {angle_combined:.4f}도") # 도75
25
26 # =====
27 # 2. 쿼터니언클래스
28 # =====
29 class Quaternion:
30     def __init__(self, w, x, y, z):
31         self.w = w
32         self.x = x
33         self.y = y
34         self.z = z
35
36     def __mul__(self, other):
37         """쿼터니언 곱셈"""
38         w1, x1, y1, z1 = self.w, self.x, self.y, self.z
39         w2, x2, y2, z2 = other.w, other.x, other.y, other.z
40         return Quaternion(
41             w1*w2 - x1*x2 - y1*y2 - z1*z2,
42             w1*x2 + x1*w2 + y1*z2 - z1*y2,
43             w1*y2 - x1*z2 + y1*w2 + z1*x2,
44             w1*z2 + x1*y2 - y1*x2 + z1*w2
45         )
46
47     def conjugate(self):
48         return Quaternion(self.w, -self.x, -self.y, -self.z)
49
50     def norm(self):
51         return np.sqrt(self.w**2 + self.x**2 + self.y**2 + self.
52             z**2)
53
54     def normalize(self):
55         n = self.norm()
56         return Quaternion(self.w/n, self.x/n, self.y/n, self.z/n
57             )
58
59     def to_array(self):
60         return np.array([self.w, self.x, self.y, self.z])
61
62     def __repr__(self):

```



```

30         return f"Q({self.w:.4f} + {self.x:.4f}i + {self.y:.4f}j
31           + {self.z:.4f}k)"
32
33 def rotation_quaternion(axis, angle_deg):
34     """회전 쿼터니언생성 """
35     axis = np.array(axis, dtype=float)
36     axis = axis / np.linalg.norm(axis)
37     half = np.radians(angle_deg) / 2
38     return Quaternion(np.cos(half),
39                       axis[0]*np.sin(half),
40                       axis[1]*np.sin(half),
41                       axis[2]*np.sin(half))
42
43 def rotate_vector(q, v):
44     """쿼터니언으로 벡터회전 : v' = q v q*"""
45     p = Quaternion(0, v[0], v[1], v[2])
46     p_rot = q * p * q.conjugate()
47     return np.array([p_rot.x, p_rot.y, p_rot.z])
48
49 # 축 z 도 90 회전
50 q = rotation_quaternion([0,0,1], 90)
51 v = np.array([1.0, 0.0, 0.0])
52 v_rot = rotate_vector(q, v)
53 print(f"\축z 도90: {v} -> {np.round(v_rot, 4)}") # [0. 1. 0.]
54
55 # 두회전합성 (축y 도90 후축 z 도90)
56 q1 = rotation_quaternion([0,1,0], 90)
57 q2 = rotation_quaternion([0,0,1], 90)
58 q_combined = q2 * q1 # 역순!
59 v_rot2 = rotate_vector(q_combined, v)
60 print(f"y90 후 z90: {v} -> {np.round(v_rot2, 4)}")
61
62 # =====
63 # 3. SLERP
64 # =====
65 def slerp(q1, q2, t):
66     """구면 선형보간 """
67     a1 = q1.to_array()
68     a2 = q2.to_array()
69     dot = np.dot(a1, a2)
70     if dot < 0: # 짧은경로선택
71         a2 = -a2
72         dot = -dot
73     if dot > 0.9995: # 거의평행 -> LERP 사용
74         result = a1 + t * (a2 - a1)
75     else:
76         omega = np.arccos(np.clip(dot, -1, 1))
77         sin_omega = np.sin(omega)
78         result = (np.sin((1-t)*omega)/sin_omega)*a1 + (np.sin(t*
79           omega)/sin_omega)*a2

```

```

108     result /= np.linalg.norm(result)
109     return Quaternion(*result)
110
111 q_start = rotation_quaternion([0,0,1], 0)
112 q_end = rotation_quaternion([0,0,1], 90)
113 q_mid = slerp(q_start, q_end, 0.5)
114 print(f"\nSLERP t=0.5: {q_mid}")
115
116 # =====
117 # 4. 쿼터니언 -> 회전행렬
118 # =====
119 def quat_to_matrix(q):
120     w, x, y, z = q.w, q.x, q.y, q.z
121     return np.array([
122         [1-2*(y*y+z*z), 2*(x*y-w*z), 2*(x*z+w*y)],
123         [2*(x*y+w*z), 1-2*(x*x+z*z), 2*(y*z-w*x)],
124         [2*(x*z-w*y), 2*(y*z+w*x), 1-2*(x*x+y*y)]
125     ])
126
127 q = rotation_quaternion([0,0,1], 90)
128 R = quat_to_matrix(q)
129 print("\n쿼터니언 -> 행렬:\n", np.round(R, 4))

```

9 연습 문제

연습 문제

1. 복소수 $z_1 = 3 + 4i$, $z_2 = 1 - 2i$ 에 대해 $z_1 z_2$, z_1/z_2 , $|z_1|$ 을 계산하라.
2. 단위 복소수 $q = e^{i\pi/4}$ 를 이용하여 점 $(2, 1)$ 을 45° 회전시킨 결과를 구하라.
3. y 축을 중심으로 60° 회전하는 쿼터니언을 구성하고, 벡터 $(0, 0, 1)$ 에 적용한 결과를 계산하라.
4. 두 쿼터니언 $q_1 = (\cos 30^\circ, 0, 0, \sin 30^\circ)$ (z 축 60°)와 $q_2 = (\cos 45^\circ, \sin 45^\circ, 0, 0)$ (x 축 90°)의 곱 $q_2 q_1$ 을 계산하고, 이 합성 회전이 $(1, 0, 0)$ 을 어디로 보내는지 구하라.
5. 단위 쿼터니언 q 에 대해 $|q^*| = |q| = 1$ 이고 $qq^* = 1$ 임을 보여라.
6. Python으로 SLERP를 구현하여 x 축 0° 에서 x 축 180° 까지 10단계로 보간된 쿼터니언들을 출력하고, 각 단계에서 벡터 $(1, 0, 0)$ 의 위치를 확인하라.

10 요약

핵심 정리

1. 복소수: $z = a + bi$ 는 2D 평면의 점; 단위 복소수 곱 = 2D 회전.
2. 쿼터니언: $q = w + xi + yj + zk$, 여기서 $i^2 = j^2 = k^2 = ijk = -1$.
3. 3D 회전: 단위 쿼터니언 $q = (\cos(\theta/2), \hat{n} \sin(\theta/2))$ 으로 축-각 표현.
4. 회전 적용: $\mathbf{v}' = q \cdot (0, \mathbf{v}) \cdot q^*$.
5. 합성: $q_2 q_1$ (역순), 역변환: $q^* = q^{-1}$ (단위 쿼터니언).
6. SLERP: 구면 위 보간으로 자연스러운 회전 애니메이션.
7. 장점: 짐벌 락 없음, SLERP 가능, 합성이 단순.
8. Python: `scipy.spatial.transform.Rotation` 또는 직접 구현.

A 오일러 공식 유도

테일러 급수를 이용한 $e^{i\theta} = \cos \theta + i \sin \theta$ 유도:

$$\begin{aligned}
 e^{i\theta} &= \sum_{n=0}^{\infty} \frac{(i\theta)^n}{n!} \\
 &= 1 + i\theta + \frac{(i\theta)^2}{2!} + \frac{(i\theta)^3}{3!} + \frac{(i\theta)^4}{4!} + \dots \\
 &= 1 + i\theta - \frac{\theta^2}{2!} - i\frac{\theta^3}{3!} + \frac{\theta^4}{4!} + \dots \\
 &= \left(1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \dots\right) + i\left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \dots\right) \\
 &= \cos \theta + i \sin \theta \quad \square
 \end{aligned}$$

B 쿼터니언 곱셈 전개

$p = (p_w + p_x i + p_y j + p_z k)$, $q = (q_w + q_x i + q_y j + q_z k)$ 의 곱을 $i^2 = j^2 = k^2 = -1$, $ij = k$, $jk = i$, $ki = j$, $ji = -k$ 등을 이용하여 전개하면:

$$\begin{aligned}
 pq &= p_w q_w + p_w q_x i + p_w q_y j + p_w q_z k \\
 &\quad + p_x q_w i + p_x q_x (-1) + p_x q_y k + p_x q_z (-j) \\
 &\quad + p_y q_w j + p_y q_x (-k) + p_y q_y (-1) + p_y q_z i \\
 &\quad + p_z q_w k + p_z q_x j + p_z q_y (-i) + p_z q_z (-1)
 \end{aligned}$$

실수부 합산:

$$w = p_w q_w - p_x q_x - p_y q_y - p_z q_z = p_w q_w - \mathbf{p} \cdot \mathbf{q}$$

i 성분:

$$x = p_w q_x + p_x q_w + p_y q_z - p_z q_y$$

이들을 정리하면 $pq = (p_w q_w - \mathbf{p} \cdot \mathbf{q}, \quad p_w \mathbf{q} + q_w \mathbf{p} + \mathbf{p} \times \mathbf{q})$

□